

ОТЧЁТ

по проектированию программного модуля

AI-МОДУЛЬ ДЛЯ ОБРАЗОВАТЕЛЬНОЙ ПЛАТФОРМЫ

Москва 2026

СОДЕРЖАНИЕ

Введение	3
1 Теоретическая часть	4
1.1 Назначение AI-модуля	4
1.2 Анализ используемых технологий	4
1.3 Требования к AI-модулю	5
2 Технологическая часть	5
2.1 Архитектура AI-модуля	5
2.2 Диаграммы проектирования	6
3 Практическая часть	10
Заключение	11
Список использованных источников	12

ВВЕДЕНИЕ

В современных условиях искусственный интеллект является важной частью цифровых сервисов, в том числе образовательных платформ. Пользователи ожидают от таких систем не только доступа к учебным материалам, но и возможности получить персональную помощь, объяснение сложной темы, разбор ошибок и рекомендации по дальнейшему обучению.

В рамках проекта AI-модуль GigaChat рассматривается как уже установленная часть образовательной платформы подготовки к экзаменам. Модуль предоставляет пользователю интеллектуального ассистента внутри курса и взаимодействует с backend через существующие API-маршруты.

Цель работы — описать установленный AI-модуль GigaChat, его назначение, архитектуру, основные сценарии использования и диаграммы проектирования.

Для достижения цели решаются следующие задачи: описать предметную область применения AI в обучении; определить требования к модулю; описать архитектуру взаимодействия frontend, backend, базы данных и GigaChat API; подготовить диаграммы Use Case, User Story Map, ERD, Sequence, Component и Activity.

1 Теоретическая часть

1.1 Назначение AI-модуля

AI-модуль предназначен для интеллектуальной поддержки пользователя в процессе изучения курса. Пользователь может задать вопрос

по теме, получить объяснение простыми словами, попросить пример, разобрать ошибку или сформировать мини-план подготовки.

Особенностью модуля является работа в контексте конкретного курса. При формировании ответа система учитывает название курса, описание, учебные материалы, слайды, теоретические блоки, примеры и практические задания. Благодаря этому ассистент отвечает не абстрактно, а применительно к содержанию выбранного курса.

1.2 Анализ используемых технологий

Для реализации AI-модуля используется GigaChat — языковая модель, доступная через внешний API. Для интеграции с backend применяется библиотека langchain-gigachat, позволяющая использовать GigaChat в архитектуре LangChain и в дальнейшем расширять модуль до полноценного AI-агента.

Backend реализован на FastAPI. Он принимает запросы от frontend, проверяет авторизацию пользователя, получает контекст курса из базы данных, формирует системный промпт и отправляет запрос к GigaChat. История сообщений сохраняется в базе данных через сущности AIChatSession и AIChatMessage.

Frontend использует React-компонент AIChatWidget, который отображает историю сообщений, поле ввода и состояние ожидания ответа. Компонент взаимодействует с backend через aiApi.ts и endpoint /api/v1/ai/ask.

1.3 Требования к AI-модулю

К функциональным требованиям относятся: приём вопроса пользователя, определение курса, получение контекста, обращение к GigaChat, возврат ответа, сохранение истории диалога и возможность повторной загрузки сообщений.

К нефункциональным требованиям относятся: безопасность хранения ключей доступа, корректная работа с авторизацией, устойчивость к ошибкам

внешнего API, понятная структура кода, возможность масштабирования и дальнейшего расширения модуля до агента с инструментами.

2 Технологическая часть

2.1 Архитектура AI-модуля

Установленный AI-модуль встроен в существующую архитектуру образовательной платформы. Пользователь отправляет сообщение из интерфейса курса. Frontend передаёт запрос в backend. Backend получает пользователя, курс и учебный контекст, после чего вызывает GigaChat через langchain-gigachat.

После генерации ответа backend сохраняет вопрос и ответ в базе данных. Затем результат возвращается на frontend и отображается в AI-чате. Такой подход позволяет не изменять основную логику клиентской части и сосредоточить AI-интеграцию в сервисном слое backend.

2.2 Диаграммы проектирования

Для описания установленного AI-модуля подготовлены диаграммы PlantUML. В документе оставлены места для вставки изображений диаграмм. После генерации PNG/SVG из PlantUML их необходимо вставить вместо соответствующих заглушек.

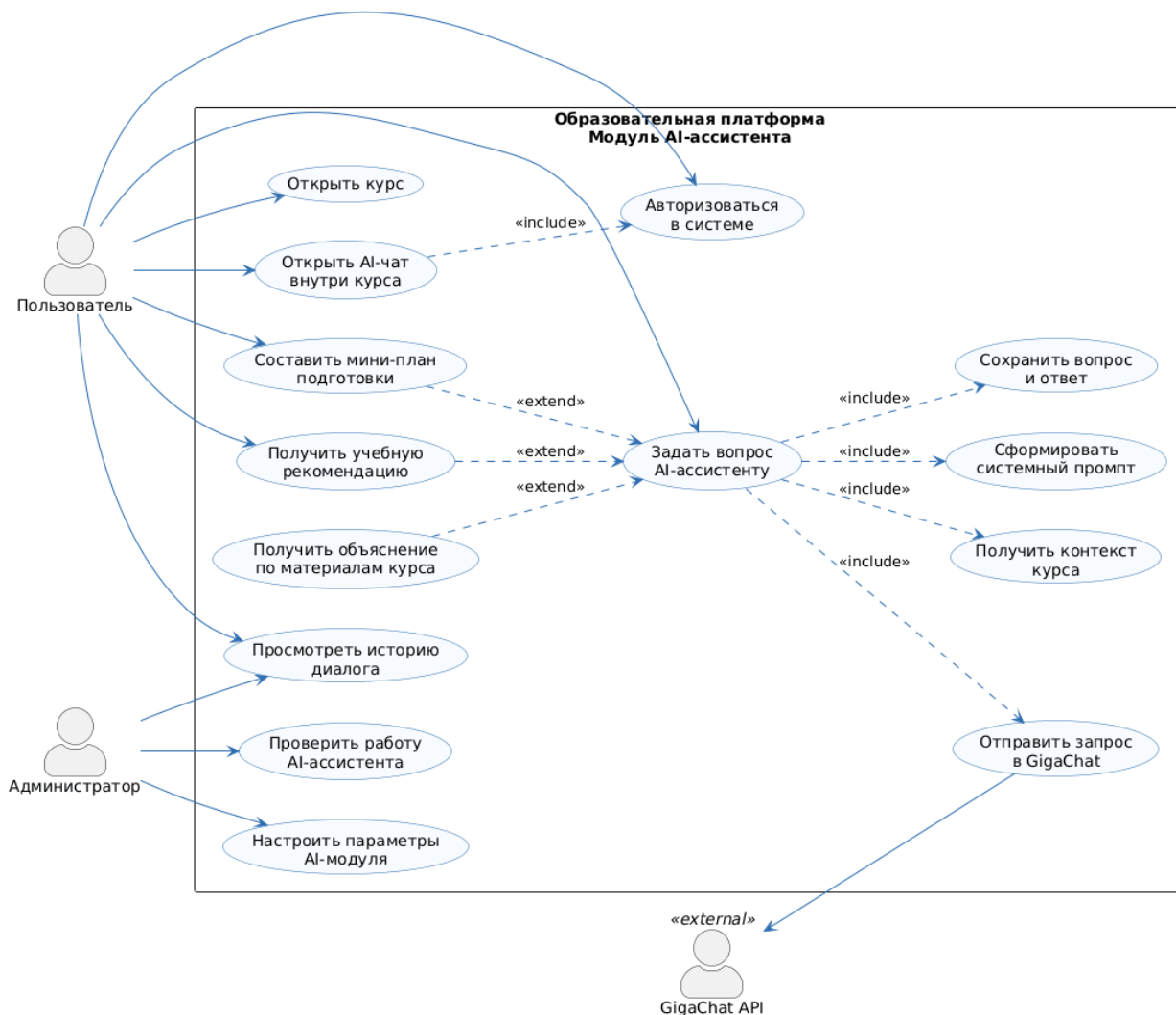


Рисунок 1 — Use Case-диаграмма AI-модуля GigaChat

Use Case-диаграмма показывает взаимодействие пользователя, администратора и внешнего GigaChat API с AI-модулем. Основной пользовательский сценарий включает открытие курса, отправку вопроса, получение ответа и просмотр истории диалога.



Рисунок 2 — User Story Map AI-модуля GigaChat

User Story Map отражает пользовательский путь от входа в систему до общения с AI-ассистентом, сохранения истории и дальнейшего развития модуля.

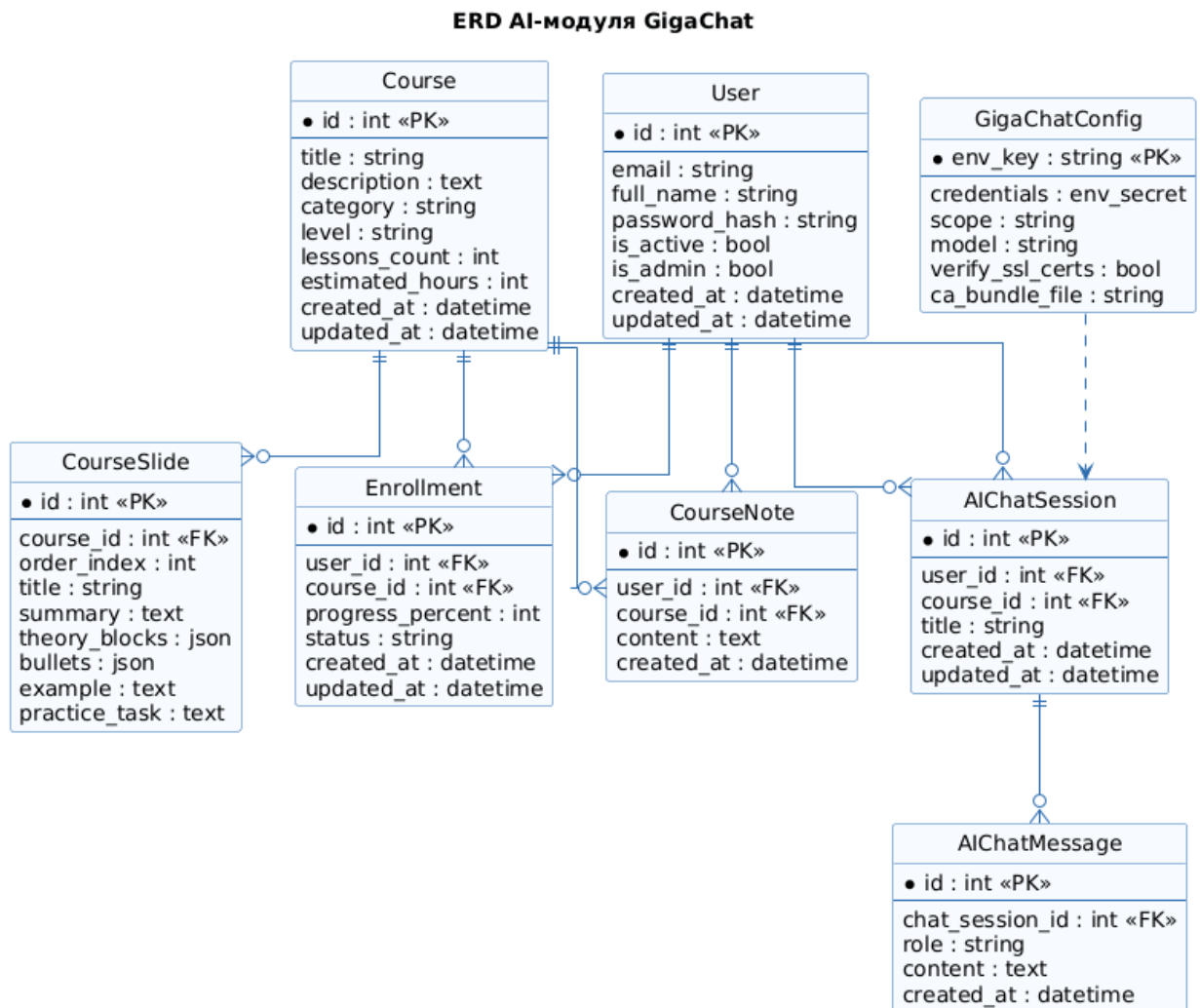


Рисунок 3 — ERD AI-модуля GigaChat

ERD описывает основные сущности, связанные с AI-модулем: пользователя, курс, слайды, сессию AI-чата и сообщения. Связи позволяют хранить историю общения в привязке к пользователю и конкретному курсу.

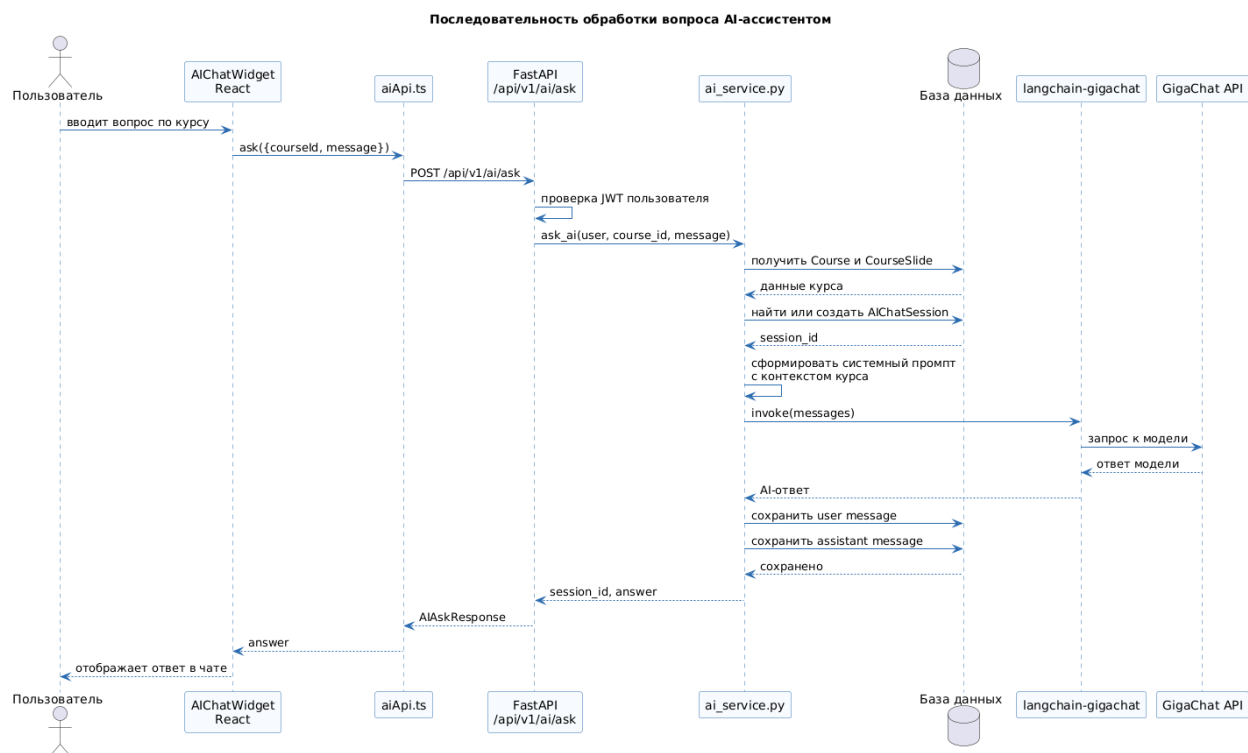


Рисунок 4 — Sequence-диаграмма обработки вопроса AI-ассистентом

Sequence-диаграмма показывает последовательность действий от ввода вопроса пользователем до получения ответа от GigaChat и сохранения результата в базе данных.

Компонентная диаграмма установленного AI-модуля GigaChat

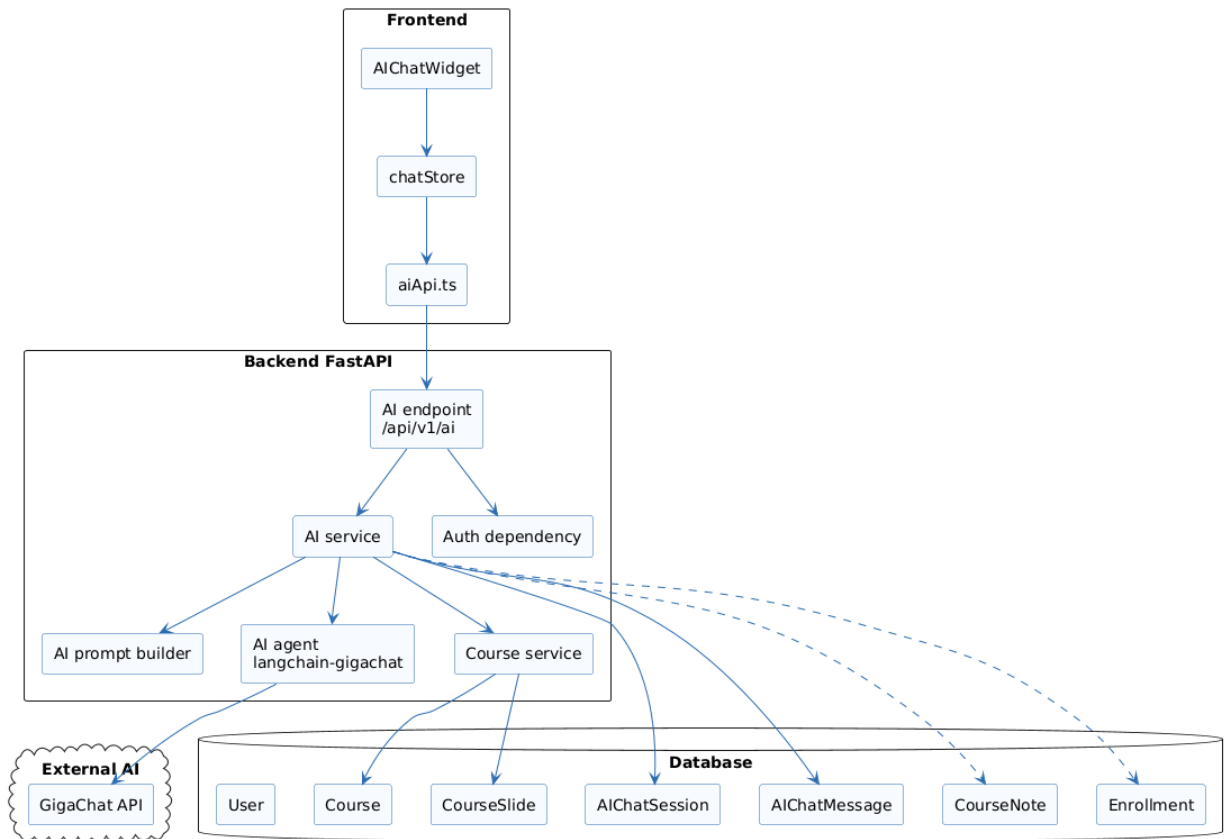


Рисунок 5 — Компонентная диаграмма установленного AI-модуля

Компонентная диаграмма показывает состав frontend, backend, базы данных и внешнего AI-сервиса, а также связи между ними.

Activity-диаграмма работы AI-модуля GigaChat

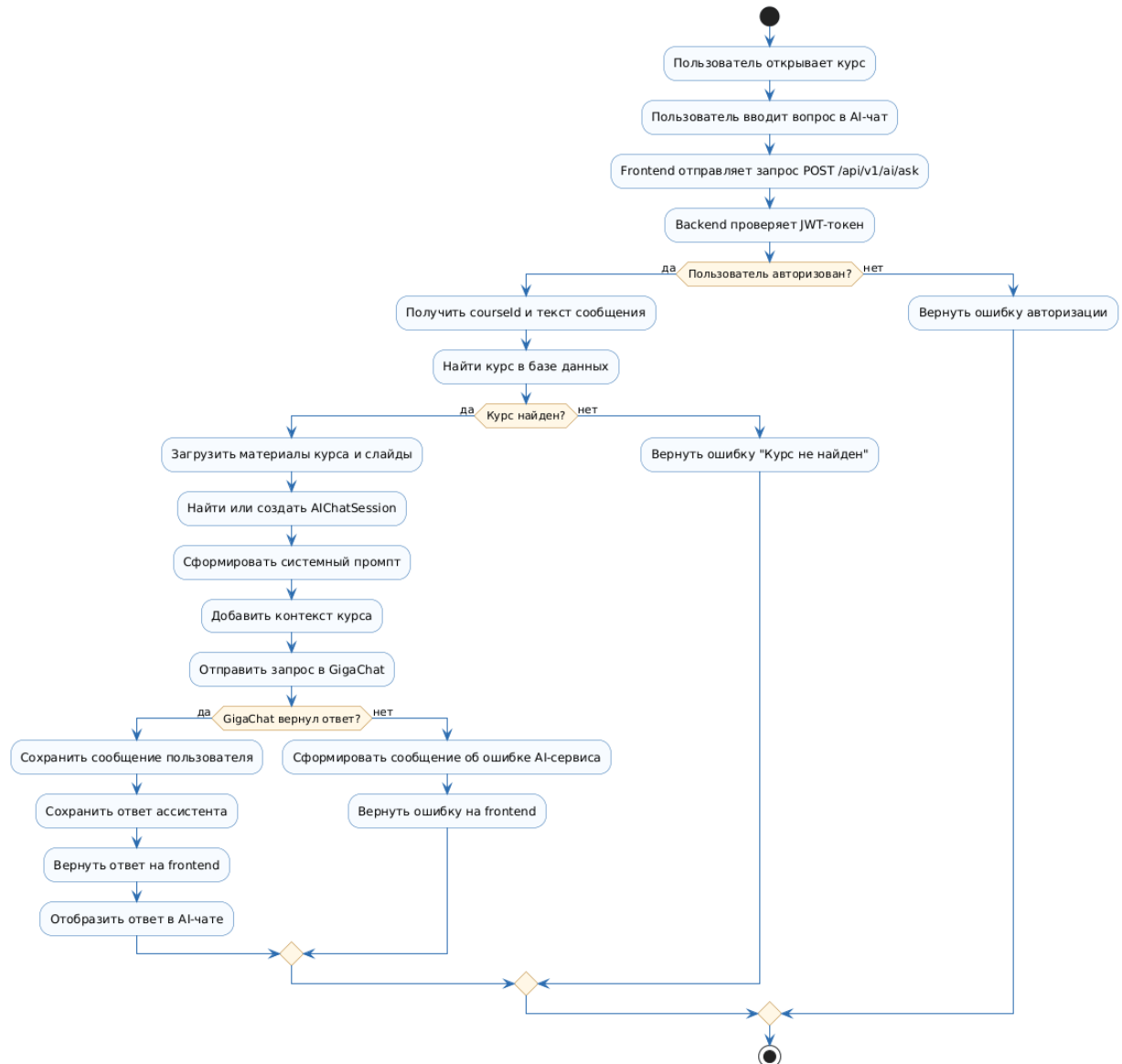


Рисунок 6 — Activity-диаграмма работы AI-модуля

Activity-диаграмма описывает общий алгоритм обработки вопроса: проверку авторизации, получение курса, формирование промпта, обращение к GigaChat, сохранение сообщений и возврат ответа пользователю.

3 Практическая часть

В установленном состоянии AI-модуль работает через endpoint `/api/v1/ai/ask`. Пользователь отправляет сообщение, backend проверяет JWT-токен, получает данные курса и формирует системный промпт. Далее запрос передаётся в GigaChat, а результат сохраняется в истории сообщений.

Для просмотра истории используется endpoint `/api/v1/ai/history`. Он возвращает список сообщений по конкретному курсу и пользователю. Это обеспечивает непрерывность диалога и позволяет пользователю возвращаться к предыдущим объяснениям.

Ключи доступа к GigaChat хранятся в переменных окружения. В конфигурации `backend` используются параметры `GIGACHAT_CREDENTIALS`, `GIGACHAT_SCOPE`, `GIGACHAT_MODEL`, `GIGACHAT_VERIFY_SSL_CERTS` и `GIGACHAT_CA_BUNDLE_FILE`. Такой подход соответствует требованиям безопасности и не допускает размещения секретных данных в исходном коде.

В дальнейшем модуль может быть расширен до AI-агента с инструментами. Агент сможет анализировать прогресс пользователя, работать с заметками, рекомендовать следующий материал и формировать индивидуальную траекторию подготовки.

ЗАКЛЮЧЕНИЕ

В результате работы описан установленный AI-модуль GigaChat для образовательной платформы. Модуль обеспечивает интеллектуальное взаимодействие с пользователем внутри курса, использует контекст учебных материалов и сохраняет историю диалога.

Использование `langchain-gigachat` позволяет не только реализовать чат-ассистента, но и подготовить основу для дальнейшего развития системы до полноценного AI-агента. Таким образом, модуль повышает интерактивность образовательной платформы и делает подготовку к экзаменам более персонализированной.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017.

2. GigaChat Python SDK: репозиторий ai-forever/gigachat. URL: <https://github.com/ai-forever/gigachat>
3. LangChain integration for GigaChat: репозиторий ai-forever/langchain-gigachat. URL: <https://github.com/ai-forever/langchain-gigachat>
4. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/>
5. PlantUML Documentation. URL: <https://plantuml.com/>

ПРИЛОЖЕНИЕ А

Перечень подготовленных PlantUML-диаграмм

- 01_ai_use_case.puml — Use Case-диаграмма AI-модуля GigaChat.
- 02_ai_user_story_map.puml — User Story Map AI-модуля GigaChat.
- 03_ai_erd.puml — ERD AI-модуля GigaChat.
- 04_ai_sequence_ask.puml — Sequence-диаграмма обработки вопроса.
- 05_ai_component.puml — компонентная диаграмма установленного AI-модуля.
- 06_ai_activity.puml — Activity-диаграмма работы AI-модуля.